



Oued-Souss Alert

Systeme de surveillance et d'alerte des crues agricoles

Rapport de projet

Prévention des inondations des zones agricoles via la surveillance du débit de l'Oued Souss

Réalisé par :

- Abdelehamid AIT BELLA
- Ouissal AIT OUARAB
- Wissal AISSAOUI

Encadré par :

- Pr. Sara EL-ATEIF

Année universitaire : 2025 – 2026

Table des matières

1	Contexte generale du projet	3
1.1	Introduction Générale	3
1.2	Problématique du projet	3
1.3	Solution proposée	3
1.4	Cahier des charges	4
2	Analyse et Conception	5
2.1	Modèle Conceptuel de Données (MCD)	5
2.2	Modèle Logique de Données (MLD)	6
3	Réalisation de l'Application	7
3.1	Conception de la base de données	7
3.1.1	Les tables	7
3.1.2	Justification 3FN	12
3.1.3	Contraintes recommandées	13
3.1.4	Optimisation(index)	13
3.1.5	Views	14
3.2	Génération Logique Métier	15
3.2.1	Triggers & Functions	15
3.2.2	Procedures	20
3.3	Présentation du projet	22
4	Conclusion	25

Résumé

Le présent rapport expose la conception et le développement du projet **Oued-Souss Alert**, un prototype de système d'information de surveillance et d'alerte précoce des crues agricoles dans la région géographique du Souss-Massa (dans le cadre de la Circulaire FLCN 2026). Ce système intelligent en temps réel, reposant sur une architecture moderne à trois niveaux (PostgreSQL, Node.js/Express, React/Vite), collecte des données simulées par des capteurs virtuels (niveaux d'eau et précipitations) pour évaluer dynamiquement le risque d'inondation. Par l'intégration d'algorithmes et de déclencheurs automatiques (triggers) au sein même de la base de données, l'application garantit une réactivité optimale permettant de déclencher instantanément des alertes en cas de dépassement des limites, offrant ainsi un outil d'aide à la décision efficace pour minimiser les dégâts matériels et environnementaux potentiels.

1 Contexte generale du projet

1.1 Introduction Générale

Le projet **Oued-Souss Alert** s'inscrit dans un contexte où les risques naturels, notamment les inondations, représentent une menace importante pour les zones agricoles de la région Souss-Massa. Les crues de l'Oued Souss peuvent causer des pertes considérables en termes de production agricole, d'infrastructures et d'impact environnemental.

Face à ces enjeux, il devient essentiel de mettre en place des systèmes intelligents capables de surveiller en temps réel les paramètres hydrologiques et météorologiques, afin d'anticiper les situations de danger.

Ce projet propose la conception et le développement d'une application web de surveillance et d'alerte des crues, basée sur une architecture moderne et intégrant des mécanismes d'automatisation au niveau de la base de données. L'objectif est de fournir un outil d'aide à la décision permettant de détecter rapidement les situations critiques et de déclencher des alertes en temps réel.

Le rapport est structuré en plusieurs chapitres présentant le contexte du projet, l'analyse et la conception du système, ainsi que la réalisation technique et les résultats obtenus.

1.2 Problématique du projet

La gestion des inondations dans les zones agricoles repose souvent sur des interventions tardives, après l'apparition des dégâts. L'absence de systèmes de surveillance en temps réel rend difficile l'anticipation des crues, ce qui expose les exploitations agricoles à des risques importants.

De plus, les données hydrologiques et météorologiques sont généralement dispersées et peu exploitées, ce qui limite la capacité à prévoir les situations critiques.

Ainsi, la problématique principale est la suivante : **Comment concevoir un système intelligent capable de surveiller en temps réel les paramètres environnementaux et de détecter automatiquement les risques d'inondation afin d'alerter les utilisateurs avant la catastrophe ?**

1.3 Solution proposée

Pour répondre à cette problématique, nous proposons le développement d'un système intelligent appelé **Oued-Souss Alert**.

Ce système repose sur :

- La collecte des données de niveau d'eau et de précipitations
- L'analyse des données en temps réel
- Le calcul automatique d'un indice de risque
- La génération automatique d'alertes en cas de danger
- L'affichage des informations sur un tableau de bord interactif

L'originalité de cette solution réside dans l'intégration de la logique métier directement au niveau de la base de données à l'aide de triggers et de procédures stockées, permettant une automatisation et une réactivité optimales.

1.4 Cahier des charges

Le système proposé doit répondre à un ensemble d'exigences fonctionnelles et non fonctionnelles afin d'assurer son efficacité et sa fiabilité. Ces exigences définissent les principales fonctionnalités attendues ainsi que les contraintes techniques à respecter.

Exigences fonctionnelles :

- Surveiller en temps réel le niveau d'eau et les précipitations
- Calculer un indice de risque de crue à partir des données collectées
- Générer automatiquement des alertes en cas de situation critique
- Afficher les zones à risque sur une carte interactive
- Gérer les utilisateurs ainsi que leurs rôles et permissions

Ces fonctionnalités permettent d'assurer un suivi précis de la situation hydrologique et d'offrir une aide efficace à la prise de décision.

Exigences non fonctionnelles :

- Assurer la sécurité et la confidentialité des données
- Garantir de bonnes performances et un temps de réponse rapide
- Offrir une interface simple, intuitive et ergonomique
- Assurer la fiabilité et la disponibilité du système

Ces contraintes visent à garantir un système robuste, performant et adapté aux besoins des utilisateurs dans des situations critiques.

2 Analyse et Conception

2.1 Modèle Conceptuel de Données (MCD)

Le Modèle Conceptuel de Données (MCD) représente la structure logique globale du système Oued-Souss Alert. Il met en évidence les principales entités du système ainsi que les relations entre elles, indépendamment de toute contrainte technique.

Les entités principales identifiées dans le système sont :

- **Utilisateur** : représente les acteurs du système (administrateur, agriculteur, opérateur).
- **Zone** : définit les zones agricoles surveillées.
- **Capteur** : dispositifs installés dans les zones pour collecter les données environnementales.
- **Mesure** : regroupe les données de mesure (niveau d'eau, pluie, etc.).
- **Indice de risque** : valeur calculée permettant d'évaluer le niveau de danger.
- **Alerte** : notifications générées lorsque le risque dépasse un seuil critique.

Ces entités sont reliées entre elles afin de modéliser le fonctionnement global du système de surveillance des crues et de gestion des alertes.

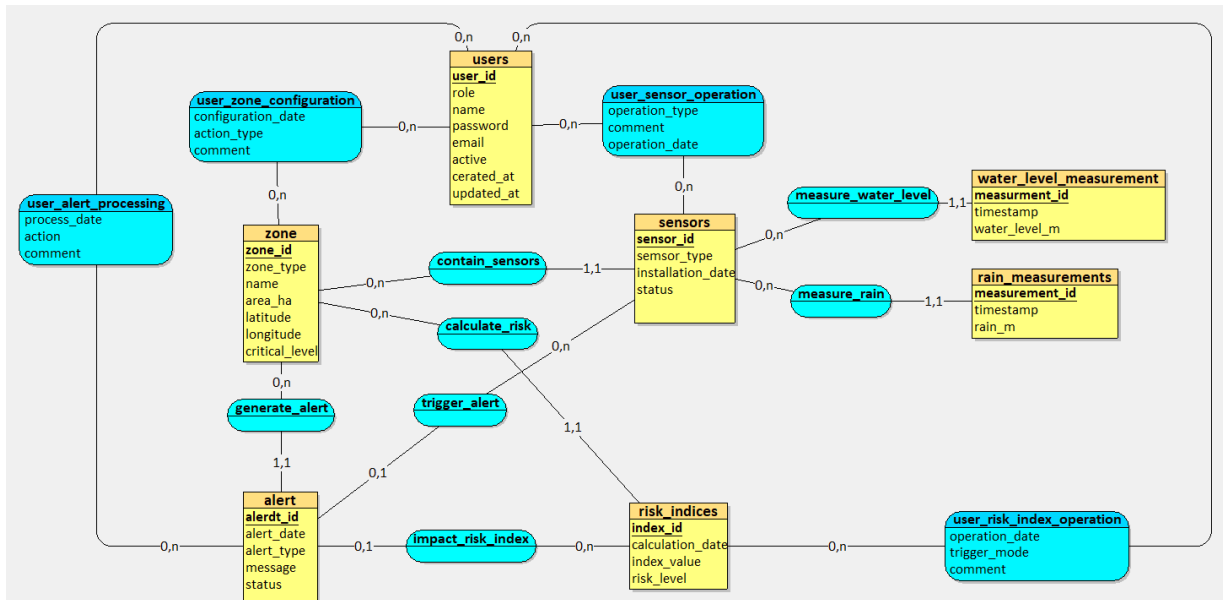


FIGURE 1 – Modèle Conceptuel de Données du système Oued-Souss Alert

2.2 Modèle Logique de Données (MLD)

Le Modèle Logique de Données (MLD) traduit le MCD en une structure relationnelle exploitable par un système de gestion de base de données. Il définit les tables, les clés primaires et étrangères ainsi que les relations entre les entités.

Le modèle relationnel du système est défini comme suit :

Afin d'assurer la traçabilité et la sécurité des opérations effectuées dans le système, plusieurs tables d'historisation des actions utilisateurs sont également définies :

```
USERS(user_id PK, role, name, password, email, active, created_at, updated_at)
ZONE(zone_id PK, zone_type, name, area_ha, latitude, longitude, critical_level)
SENSORS(sensor_id PK, sensor_type, installation_date, status, zone_id FK)
WATER_LEVEL_MEASUREMENT(measurement_id PK, timestamp, water_level_m, sensor_id FK UNIQUE)
RAIN_MEASUREMENTS(measurement_id PK, timestamp, rain_m, sensor_id FK UNIQUE)
RISK_INDICES(index_id PK, calculation_date, index_value, risk_level, sensor_id FK)
ALERT(alert_id PK, alert_date, alert_type, message, status, zone_id FK, index_id FK NULL)
USER_ZONE_CONFIGURATION(user_id FK, zone_id FK, configuration_date, action_type, comment, PK(user_id, zone_id, configuration_date))
USER_SENSOR_OPERATION(user_id FK, sensor_id FK, operation_type, comment, operation_date, PK(user_id, sensor_id, operation_date))
USER_ALERT_PROCESSING(user_id FK, alert_id FK, process_date, action, comment, PK(user_id, alert_id, process_date))
USER_RISK_INDEX_OPERATION(user_id FK, index_id FK, operation_date, trigger_mode, comment, PK(user_id, index_id, operation_date))
```

FIGURE 2 – Modèle Logique de Données du système Oued-Souss Alert

3 Réalisation de l'Application

3.1 Conception de la base de données

3.1.1 Les tables

– **Prompt :**

Tu es un Expert en Base de Données PostgreSQL. Nous développons le projet "Oued-Souss Alert", une application de monitoring des inondations avec un backend Node.js et un frontend contenant une carte interactive en temps réel. Le projet vise à prévenir les inondations agricoles dans la région Souss-Massa en surveillant le débit de l'Oued Souss et en générant des alertes automatiques. Il doit surveiller le niveau d'eau en temps réel, analyser l'historique des pluies, calculer un indice de risque de crue, générer des alertes automatiques et afficher les zones menacées sur un dashboard interactif. Elaborer la conception de la base de données en respectant la normalisation 3FN.

– **Résultats :**

FIGURE 3 – Table users

```
CREATE TABLE users (  
  user_id      SERIAL PRIMARY KEY,  
  name         VARCHAR(100) NOT NULL,  
  email        VARCHAR(150) NOT NULL UNIQUE,  
  password     VARCHAR(255) NOT NULL, -- bcrypt hashed  
  role         VARCHAR(20)  NOT NULL CHECK (role IN ('admin', 'operator', 'reader', 'security')),  
  active       BOOLEAN     DEFAULT TRUE,  
  created_at   TIMESTAMP   DEFAULT NOW(),  
  updated_at   TIMESTAMP   DEFAULT NOW()  
);
```

La table **users** stocke les informations relatives aux utilisateurs du système. Elle contient des données d'identification comme le nom, l'email (qui doit être unique), ainsi que le mot de passe stocké sous forme chiffrée pour garantir la sécurité. Chaque utilisateur possède un rôle spécifique (administrateur, opérateur, lecteur ou agent de sécurité), ce qui permet de gérer les droits d'accès au système. Un champ `active` indique si le compte est actif ou non, tandis que les champs `created_at` et `updated_at` permettent de tracer la création et les modifications du compte.

FIGURE 4 – Table zones

```
-- Geographical zones  
CREATE TABLE zones (  
  zone_id      SERIAL PRIMARY KEY,  
  name         VARCHAR(100) NOT NULL,  
  zone_type    VARCHAR(50)  CHECK (zone_type IN ('agricultural', 'urban', 'mixed')),  
  area_ha      DECIMAL(10,2),  
  latitude     DECIMAL(9,6),  
  longitude    DECIMAL(9,6),  
  critical_level NUMERIC     NOT NULL CHECK (critical_level > 0)  
);
```

– **Hallucination :** Ne pas ajouter l'attribut `critical_level`

La table **zone** représente les différentes zones géographiques surveillées par le système. Chaque zone est définie par un nom, un type (agricole, urbain ou mixte), ainsi que des caractéristiques physiques comme la superficie en hectares et ses coordonnées géographiques (latitude et longitude). Elle inclut également un niveau critique, qui correspond à un seuil de référence utilisé pour évaluer les risques d'inondation dans cette zone. Cette table est centrale car elle relie les capteurs et les indices de risque. Le seuil critique représente le niveau d'eau à partir duquel une zone est considérée comme menacée par une inondation. Elle représente un niveau d'eau mesuré en mètres. Il est physiquement impossible qu'un seuil d'alerte soit négatif. La contrainte CHECK (`critical_level > 0`) garantit l'intégrité des données en empêchant l'insertion d'un seuil négatif ou nul et éviter les erreurs dans les calculs de risque. Chaque zone a une altitude différente, une topographie différente et une capacité de drainage différente donc une seuil de danger différente.

FIGURE 5 – Table sensors

```
CREATE TABLE sensors (
  sensor_id      SERIAL PRIMARY KEY,
  zone_id        INTEGER      NOT NULL,
  sensor_type     VARCHAR(50)  NOT NULL CHECK (sensor_type IN ('water_level', 'rain')),
  installation_date DATE,
  status          VARCHAR(20)  DEFAULT 'active' CHECK (status IN ('active', 'maintenance', 'offline')),
  CONSTRAINT fk_sensors_zone
    FOREIGN KEY (zone_id) REFERENCES zones(zone_id)
    ON DELETE CASCADE
);
```

La table **sensors** représente les dispositifs physiques installés sur le terrain pour collecter les données hydrologiques. Chaque capteur est associé à une zone spécifique via une clé étrangère (`zone_id`). Les capteurs peuvent être de différents types (niveau d'eau ou de pluie). Cette table contient également des informations sur la date d'installation et le statut du capteur (actif, en maintenance, hors service), ce qui permet d'assurer un suivi technique du matériel. La relation entre les zones et les capteurs est de type un-à-plusieurs : une zone peut contenir plusieurs capteurs, mais chaque capteur appartient à une seule zone.

La contrainte **fk_sensors_zone** établit une relation d'intégrité référentielle entre la table capteurs et la table zones. Elle garantit qu'un capteur ne peut être associé qu'à une zone existante. Si une zone est supprimée, tous les capteurs qui lui sont associés seront automatiquement supprimés grâce à l'option `ON DELETE CASCADE`. Cela évite la présence de capteurs orphelins dans la base de données.

FIGURE 6 – Table water_level_measurements

```
CREATE TABLE water_level_measurements (
  measurement_id SERIAL PRIMARY KEY,
  sensor_id       INTEGER      NOT NULL,
  timestamp       TIMESTAMP    NOT NULL,
  water_level_m   NUMERIC(5,2) NOT NULL,
  CONSTRAINT fk_water_level_sensor
    FOREIGN KEY (sensor_id)
    REFERENCES sensors(sensor_id)
    ON DELETE CASCADE
);
```

La table **water_level_measurements** stocke l'ensemble des mesures historiques du niveau d'eau relevées par les capteurs. Chaque enregistrement contient un identifiant unique, une

référence au capteur ayant effectué la mesure, la date et l'heure exactes de la collecte ainsi que la valeur mesurée en mètres. Cette table est essentielle pour l'analyse temporelle, la visualisation des tendances, la détection des crues et le calcul des indices de risque.

La contrainte **fk_water_level_sensor** assure que chaque mesure de niveau d'eau est associée à un capteur existant. Une mesure ne peut donc pas être enregistrée pour un capteur inexistant. Si un capteur est supprimé, toutes les mesures associées seront également supprimées.

FIGURE 7 – Table rain_measurements

```
-- Rain measurements
CREATE TABLE rain_measurements (
  measurement_id SERIAL PRIMARY KEY,
  sensor_id      INTEGER      NOT NULL,
  timestamp      TIMESTAMP    NOT NULL,
  rain_mm        NUMERIC(5,2) NOT NULL,
  CONSTRAINT fk_rain_sensor
    FOREIGN KEY (sensor_id)
    REFERENCES sensors(sensor_id)
    ON DELETE CASCADE
);
```

La table **rain_measurements** enregistre les données relatives aux précipitations. Sa structure est similaire à celle des mesures de niveau d'eau, mais elle stocke la quantité de pluie mesurée en millimètres. Les données de pluie jouent un rôle déterminant dans l'évolution du niveau d'eau et permettent d'anticiper les risques d'inondation. Ces données sont essentielles pour le calcul des indices de risque en corrélant les données de pluie avec les niveaux d'eau, le système peut établir des modèles prédictifs plus fiables et améliorer la précision des alertes.

La contrainte **fk_rain_sensor** garantit que chaque mesure de précipitation est associée à un capteur existant. Si un capteur est supprimé, toutes les mesures de pluie associées seront également supprimées automatiquement.

FIGURE 8 – Table risk_indices

```
CREATE TABLE risk_indices (
  index_id      SERIAL PRIMARY KEY,
  zone_id       INTEGER      NOT NULL,
  calculation_date TIMESTAMP  NOT NULL,
  index_value    NUMERIC(5,2) NOT NULL,
  risk_level     VARCHAR(20)  CHECK (risk_level IN ('LOW', 'MEDIUM', 'HIGH', 'CRITICAL')),
  CONSTRAINT fk_risk_zone
    FOREIGN KEY (zone_id)
    REFERENCES zones(zone_id)
    ON DELETE CASCADE
);
```

La table **risk_indices** contient les résultats des calculs d'indices de risque effectués périodiquement. Chaque indice est associé à une zone donnée et à une date précise de calcul. La valeur de l'indice est généralement comprise entre 0 et 1 et permet de classifier le niveau de risque (faible / modéré / élevé). Le stockage de ces indices permet de conserver un historique analytique, facilitant les comparaisons temporelles, les audits et l'évaluation des performances du système de prédiction. Cette table permet de synthétiser les données issues des capteurs afin de faciliter la prise de décision.

La contrainte **fk_risk_zone** assure que chaque indice de risque calculé est associé à une zone existante. Un indice de risque ne peut donc pas exister sans zone correspondante. Si une zone est supprimée, tous les indices de risque associés sont également supprimés. Cela évite de conserver des indices qui ne correspondent plus à aucune zone.

FIGURE 9 – Table alerts

```
CREATE TABLE alerts (  
  alert_id      SERIAL PRIMARY KEY,  
  zone_id       INTEGER      NOT NULL,  
  index_id      INTEGER,  
  sensor_id     INTEGER,  
  alert_date    TIMESTAMP    NOT NULL,  
  alert_type    VARCHAR(50)  CHECK (alert_type IN ('FLOOD', 'HEAVY_RAIN', 'LEVEL_EXCEEDED')),  
  message       TEXT,  
  status        VARCHAR(20)  DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE', 'RESOLVED')),  
  CONSTRAINT fk_alerts_zone  
    FOREIGN KEY (zone_id)  
    REFERENCES zones(zone_id)  
    ON DELETE CASCADE,  
  CONSTRAINT fk_alerts_index  
    FOREIGN KEY (index_id)  
    REFERENCES risk_indices(index_id)  
    ON DELETE SET NULL,  
  CONSTRAINT fk_alerts_sensor  
    FOREIGN KEY (sensor_id)  
    REFERENCES sensors(sensor_id)  
    ON DELETE SET NULL  
);
```

La table **alerts** regroupe les alertes générées par le système en cas de situation anormale ou dangereuse. Chaque alerte est associée à une zone, et éventuellement à un capteur ou à un indice de risque. Elle contient la date de l’alerte, son type (inondation, forte pluie ou dépassement de seuil), ainsi qu’un message descriptif. Le statut de l’alerte permet de savoir si elle est toujours active ou déjà résolue. Un message descriptif et un statut indiquant si l’alerte est active ou résolue. Cette table joue un rôle crucial dans la communication des risques aux utilisateurs finaux, notamment via le tableau de bord interactif. Elle permet également de conserver une trace historique des événements critiques.

La contrainte **fk_alerts_zone** garantit que chaque alerte est associée à une zone existante. Une alerte ne peut donc pas être créée pour une zone inexistante. Si une zone est supprimée, toutes les alertes qui lui sont associées seront automatiquement supprimées. Cela permet d’éviter les incohérences dans la gestion des alertes.

La contrainte **fk_alerts_index** relie une alerte à un indice de risque spécifique. Toutefois, si l’indice de risque correspondant est supprimé, la valeur `index_id` dans la table `alerts` est automatiquement remplacée par `NULL` grâce à l’option `ON DELETE SET NULL`. Cette stratégie est utilisée car une alerte peut rester pertinente même si l’indice de risque qui l’a générée est supprimé de l’historique. Cela permet de conserver l’historique des alertes tout en évitant les références invalides.

FIGURE 10 – Table user_zone_configuration

```
CREATE TABLE user_zone_configuration (
  user_id      INTEGER      NOT NULL,
  zone_id      INTEGER      NOT NULL,
  configuration_date TIMESTAMP NOT NULL DEFAULT NOW(),
  action_type  VARCHAR(50)  NOT NULL,
  comment      TEXT,
  PRIMARY KEY (user_id, zone_id, configuration_date),
  CONSTRAINT fk_uzc_user FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
  CONSTRAINT fk_uzc_zone FOREIGN KEY (zone_id) REFERENCES zones(zone_id) ON DELETE CASCADE
);
```

La table **user_zone_configuration** est une table d'association qui enregistre les actions effectuées par les utilisateurs sur les zones. Elle permet de garder un historique des configurations ou modifications apportées, avec la date de l'action, le type d'action réalisé et éventuellement un commentaire. La clé primaire est composée de plusieurs attributs afin de permettre plusieurs actions dans le temps pour un même utilisateur et une même zone.

FIGURE 11 – Table user_sensor_operations

```
CREATE TABLE user_sensor_operations (
  user_id      INTEGER      NOT NULL,
  sensor_id    INTEGER      NOT NULL,
  operation_date TIMESTAMP   NOT NULL DEFAULT NOW(),
  operation_type VARCHAR(50) NOT NULL,
  comment      TEXT,
  PRIMARY KEY (user_id, sensor_id, operation_date),
  CONSTRAINT fk_uso_user FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
  CONSTRAINT fk_uso_sensor FOREIGN KEY (sensor_id) REFERENCES sensors(sensor_id) ON DELETE CASCADE
);
```

La table **user_sensor_operations** trace les opérations réalisées par les utilisateurs sur les capteurs. Elle enregistre quel utilisateur a effectué une action sur quel capteur, à quelle date, ainsi que le type d'opération (par exemple maintenance ou activation). Un champ commentaire permet d'ajouter des détails supplémentaires. Cette table est utile pour assurer la traçabilité des interventions techniques.

FIGURE 12 – Table user_alerts_processing

```
CREATE TABLE user_alerts_processing (
  user_id      INTEGER      NOT NULL,
  alert_id     INTEGER      NOT NULL,
  process_date  TIMESTAMP   NOT NULL DEFAULT NOW(),
  action       VARCHAR(50)  NOT NULL,
  comment      TEXT,
  PRIMARY KEY (user_id, alert_id, process_date),
  CONSTRAINT fk_uap_user FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
  CONSTRAINT fk_uap_alert FOREIGN KEY (alert_id) REFERENCES alerts(alert_id) ON DELETE CASCADE
);
```

La table **user_alerts_processing** conserve l'historique des traitements des alertes par les utilisateurs. Elle permet de savoir qui a pris en charge une alerte, à quel moment, et quelle action a été effectuée (validation, résolution, etc.). Cette traçabilité est importante pour analyser la gestion des incidents et améliorer les procédures.

FIGURE 13 – Table user_alerts_processing

```
CREATE TABLE user_risk_index_operations (
  user_id      INTEGER      NOT NULL,
  index_id     INTEGER      NOT NULL,
  operation_date  TIMESTAMP  NOT NULL DEFAULT NOW(),
  trigger_mode  VARCHAR(50) NOT NULL,
  comment      TEXT,
  PRIMARY KEY (user_id, index_id, operation_date),
  CONSTRAINT fk_urx_user FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
  CONSTRAINT fk_urx_index FOREIGN KEY (index_id) REFERENCES risk_indices(index_id) ON DELETE CASCADE
);
```

La table **user_alerts_processing** enregistre les interactions des utilisateurs avec les indices de risque, notamment les déclenchements manuels ou les recalculs. Elle contient des informations sur l'utilisateur, l'indice concerné, la date de l'opération, ainsi que le mode de déclenchement. Cela permet de distinguer les calculs automatiques des interventions humaines.

– **Prompt :**

Avec le temps, les tables **water_level_measurements** et **rain_measurements** peuvent contenir des millions de lignes. Cela peut ralentir les requêtes. Créer des tables d'archive pour stocker les anciennes mesures.

– **Résultats :**

FIGURE 14 – Table archives

```
-- Archive of water level measurements
CREATE TABLE water_level_measurements_archive AS TABLE water_level_measurements WITH NO DATA;

-- Archive of rain measurements
CREATE TABLE rain_measurements_archive AS TABLE rain_measurements WITH NO DATA;
```

Ces tables d'archive sont utilisées pour stocker les anciennes données de mesures afin d'alléger les tables principales et d'améliorer les performances du système. Elles reprennent la même structure que les tables de mesures correspondantes, mais sans contraintes de clés étrangères, ce qui facilite l'archivage et la gestion des données historiques. Elles sont particulièrement utiles pour conserver des volumes importants de données sur le long terme.

3.1.2 Justification 3FN

- **1FN** : Tous les attributs des tables contiennent des valeurs atomiques. Chaque colonne représente une seule information. Par exemple dans la table **mesures_niveau_eau**, chaque ligne capteur_id, date_mesure et niveau_eau correspond à une seule mesure.
- **2FN** : Toutes les tables possèdent des clés primaires simples et tous ses autres attributs dépendent entièrement de ces clés primaires. Il n'existe aucune dépendance partielle.
- **3FN** : Aucune dépendance transitive. Aucun attribut non-clé ne dépend d'un autre attribut non-clé. Par exemple dans la table **zones** les attributs seuil_critique, superficie ou type_zone dépendent directement de id et non entre eux.

3.1.3 Contraintes recommandées

```
-- Risk index value between 0 and 1
ALTER TABLE risk_indices
  ADD CONSTRAINT chk_index_value
  CHECK (index_value >= 0 AND index_value <= 1);

-- Zone area positive if set
ALTER TABLE zones
  ADD CONSTRAINT chk_area_positive
  CHECK (area_ha IS NULL OR area_ha > 0);

-- Sensor installation date cannot be in the future
ALTER TABLE sensors
  ADD CONSTRAINT chk_sensor_installation_date
  CHECK (installation_date IS NULL OR installation_date <= CURRENT_DATE);

-- Water level measurement timestamp cannot be in the future (+1s tolerance)
ALTER TABLE water_level_measurements
  ADD CONSTRAINT chk_water_measurement_date
  CHECK (timestamp <= NOW() + INTERVAL '1 second');

-- Rain measurement timestamp cannot be in the future (+1s tolerance)
ALTER TABLE rain_measurements
  ADD CONSTRAINT chk_rain_measurement_date
  CHECK (timestamp <= NOW() + INTERVAL '1 second');

-- Alert timestamp cannot be in the future (+1s tolerance)
ALTER TABLE alerts
  ADD CONSTRAINT chk_alert_date
  CHECK (alert_date <= NOW() + INTERVAL '1 second');

-- Drop FK in case PostgreSQL created implicit ones (precaution)
ALTER TABLE water_level_measurements_archive
  DROP CONSTRAINT IF EXISTS fk_water_level_sensor;

ALTER TABLE rain_measurements_archive
  DROP CONSTRAINT IF EXISTS fk_rain_sensor;
```

On évite les clés étrangères (FK) dans les tables d'archive pour éviter les erreurs lors de suppression, garder les données historiques même si les données liées disparaissent et améliorer les performances.

3.1.4 Optimisation(index)

– Prompt :

Afin d'optimiser les performances de la base de données, créer des index sur les colonnes fréquemment utilisées dans les opérations de recherche.

– Résultats :

```

-- Sensors by zone
CREATE INDEX idx_sensors_zone ON sensors(zone_id);

-- Water measurements by sensor + timestamp desc
CREATE INDEX idx_water_sensor_date ON water_level_measurements(sensor_id, timestamp DESC);

-- Rain measurements by sensor + timestamp desc
CREATE INDEX idx_rain_sensor_date ON rain_measurements(sensor_id, timestamp DESC);

-- Risk indices by zone + date desc
CREATE INDEX idx_risk_zone_date ON risk_indices(zone_id, calculation_date DESC);

-- Alerts by zone + date desc
CREATE INDEX idx_alerts_zone_date ON alerts(zone_id, alert_date DESC);

-- Alerts status (filter active)
CREATE INDEX idx_alerts_status ON alerts(status);

-- Alerts by sensor
CREATE INDEX idx_alerts_sensor ON alerts(sensor_id);

```

idx_sensors_zone Il permet d’optimiser les requêtes qui cherchent tous les capteurs appartenant à une zone spécifique. Dans un système de surveillance hydrologique, il est fréquent d’afficher les capteurs d’une zone donnée afin de consulter leurs mesures ou leur état de fonctionnement.

idx_water_sensor_date Il est recommandé car les requêtes analytiques sur les données hydrologiques consistent souvent à récupérer les dernières mesures d’un capteur spécifique. Grâce à cet index on peut localiser rapidement les mesures d’un capteur et les retourner directement dans l’ordre chronologique décroissant, sans effectuer un tri supplémentaire.

idx_rain_sensor_date Il est recommandé car les analyses météorologiques et hydrologiques nécessitent souvent l’accès aux précipitations récentes enregistrées par un capteur. Il permet de filtrer rapidement les données d’un capteur et de récupérer les dernières mesures sans parcourir toute la table.

idx_risk_zone_date permet d’optimiser les requêtes qui récupèrent les indices de risque pour une zone spécifique, en particulier les indices les plus récents. Dans un tableau de bord de surveillance, il est fréquent d’afficher l’indice de risque actuel d’une zone.

idx_alerts_zone_date Permet d’optimiser les requêtes qui affichent les alertes associées à une zone donnée, souvent triées par date pour consulter les alertes les plus récentes dans les tableaux de bord et les systèmes de suivi.

idx_alerts_status Cet index est créé sur l’attribut statut, qui indique si une alerte est active ou résolue. Il est recommandé car les systèmes de surveillance filtrent souvent les alertes selon leur statut, par exemple pour afficher uniquement les alertes actives.

idx_alerts_sensor permet d’accélérer toutes les requêtes qui filtrent ou utilisent sensor_id dans la table alerts. Il est utile lors de la recherche par capteur.

3.1.5 Views

– Prompt 1 :

Je veux que tu me crées une vue SQL nommée risk_summary_view. Cette vue servira de point d’entrée unique pour notre backend afin d’alimenter le dashboard cartographique. Elle doit consolider l’état actuel de chaque zone.

– Résultats :

FIGURE 15 – Risk summary view

```

CREATE OR REPLACE VIEW risk_summary_view AS
SELECT
  z.zone_id,
  z.name AS zone_name,
  z.zone_type,
  z.latitude,
  z.longitude,
  z.critical_level,

  -- Latest water level measurement (most recent active sensor)
  wlm.water_level_m AS last_water_level_m,
  wlm.timestamp AS last_measurement_at,
  wlm.sensor_id AS last_sensor_id,

  -- Percentage of the critical threshold reached (useful for the frontend gauge)
  ROUND(
    (wlm.water_level_m / NULLIF(z.critical_level, 0)) * 100, 1
  ) AS critical_level_pct,

  -- Latest calculated risk index
  ri.index_value AS last_risk_index,
  ri.risk_level AS last_risk_level,
  ri.calculation_date AS last_index_calculated_at,

  -- Current active alert (NULL if none)
  al.alert_id AS active_alert_id,
  al.alert_type AS active_alert_type,
  al.alert_date AS active_alert_since,
  al.message AS active_alert_message
FROM zones z

-- Latest water level measurement (DISTINCT ON to keep 1 row per zone)
LEFT JOIN LATERAL (
  SELECT wl.water_level_m, wl.timestamp, wl.sensor_id
  FROM water_level_measurements wl
  JOIN sensors s ON s.sensor_id = wl.sensor_id
  WHERE s.zone_id = z.zone_id
  AND s.status = 'active'
  ORDER BY wl.timestamp DESC
  LIMIT 1
) wlm ON TRUE

-- Latest calculated risk index
LEFT JOIN LATERAL (
  SELECT r.index_value, r.risk_level, r.calculation_date
  FROM risk_indices r
  WHERE r.zone_id = z.zone_id
  ORDER BY r.calculation_date DESC
  LIMIT 1
) ri ON TRUE

-- Current active alert (NULL if none)
LEFT JOIN LATERAL (
  SELECT a.alert_id, a.alert_type, a.alert_date, a.message
  FROM alerts a
  WHERE a.zone_id = z.zone_id
  AND a.status = 'ACTIVE'
  ORDER BY a.alert_date DESC
  LIMIT 1
) al ON TRUE;

```

Permet l'optimisation des performances pour la Carte Interactive (Dashboard), sans la vue, le backend Node.js devrait faire plusieurs requêtes séparées (une pour récupérer les zones, une autre pour chercher la dernière mesure de chaque capteur, une autre pour le dernier indice de risque, etc.) ou écrire une requête SQL géante et difficile à maintenir à chaque appel de l'API. Ainsi la simplification drastique du code Backend (Node.js) en encapsulant logique complexe. Et permet de formater les données sur mesure pour l'Interface Utilisateur (Frontend).

3.2 Génération Logique Métier

3.2.1 Triggers & Functions

– **Note :**

Dans **PostgreSQL** il est obligatoire de créer une fonction séparée avant de créer le trigger. Le trigger détecte l'événement et doit appeler la fonction qui exécute la logique.

– **Prompt 1 :**

Créer un Trigger qui garantir l'intégrité des données provenant des capteurs hydrologiques en vérifiant la valeur du niveau d'eau.

– **Résultats :**

FIGURE 16 – Trigger de validation du niveau d'eau

```

CREATE TRIGGER trg_check_water_level
BEFORE INSERT ON water_level_measurements
FOR EACH ROW
EXECUTE FUNCTION check_valid_water_level();

```

```

CREATE OR REPLACE FUNCTION check_valid_water_level()
RETURNS TRIGGER AS $$
BEGIN
  -- Reject outlier values (e.g., -50m sent by failing sensor)
  IF NEW.water_level_m < 0 OR NEW.water_level_m > 20 THEN
    RAISE EXCEPTION
      'Invalid water level value: % m. Accepted range: [0, 20]',
      NEW.water_level_m;
  END IF;

  RETURN NEW; -- allow insertion
END;
$$ LANGUAGE plpgsql;

```

(a) Trigger

(b) Function

On crée un trigger attaché à la table `water_level_measurements`, donc il se déclenche seulement

lorsqu'une nouvelle mesure est insérée dans cette table.

Le trigger s'exécute avant l'insertion (BEFORE INSERT) afin de vérifier la validité de la donnée avant qu'elle soit enregistrée dans la base.

Lorsqu'une insertion est effectuée, le trigger appelle la fonction `check_valid_water_level()` qui vérifie si la valeur `niveau_eau` est comprise dans un intervalle réaliste (entre 0 et 20 mètres).

Si la valeur est inférieure à 0 ou supérieure à 20, la fonction déclenche une erreur avec `RAISE EXCEPTION`. Dans ce cas l'insertion est annulée.

Sinon la fonction retourne `NEW`, qui représente la nouvelle ligne à insérer, et l'insertion se poursuit normalement.

Ce mécanisme permet de garantir l'intégrité des données provenant des capteurs hydrologiques.

– **Prompt 2 :**

Créer un Trigger qui garantir l'intégrité des données provenant des capteurs hydrologiques en vérifiant la valeur de la quantité de pluie.

– **Résultats :**

FIGURE 17 – Trigger de validation de la quantité de pluie

```
CREATE TRIGGER trg_check_rain
BEFORE INSERT ON rain_measurements
FOR EACH ROW
EXECUTE FUNCTION check_valid_rain();
```

(a) Trigger

```
CREATE OR REPLACE FUNCTION check_valid_rain()
RETURNS TRIGGER AS $$
BEGIN
    -- Reject negative or physically impossible values
    IF NEW.rain_mm < 0 OR NEW.rain_mm > 500 THEN
        RAISE EXCEPTION
            'Invalid rain value: % mm. Accepted range: [0, 500]',
            NEW.rain_mm;
    END IF;

    RETURN NEW; -- allow insertion
END;
$$ LANGUAGE plpgsql;
```

(b) Function

Ce trigger est attaché à la table `rain_measurements`. Il se déclenche automatiquement avant chaque insertion d'une mesure de pluie.

Lorsqu'une nouvelle mesure est insérée, le trigger appelle la fonction `check_valid_rain()` qui vérifie que la quantité de pluie est réaliste.

Si la valeur est négative ou supérieure à 500 mm, la fonction génère une erreur et l'insertion est annulée.

Sinon, la fonction retourne `NEW` pour autoriser l'insertion de la mesure.

Ce trigger permet d'éviter les données incohérentes provenant des capteurs météorologiques.

– **Prompt 3 :**

Créer un Trigger qui génère une alerte automatique si l'indice de risque devient critique.

– **Résultats :**

FIGURE 18 – Génération automatique d’alerte critique

```
CREATE TRIGGER trg_generate_critical_alert
AFTER INSERT ON risk_indices -- executed A
FOR EACH ROW
EXECUTE FUNCTION generate_critical_alert();
```

(a) Trigger

```
CREATE OR REPLACE FUNCTION generate_critical_alert()
RETURNS TRIGGER AS $$
DECLARE
    v_sensor_id INT;
BEGIN
    -- Check if the calculated risk level is critical
    IF NEW.risk_level = 'CRITICAL' THEN
        -- Do not create a duplicate if an ACTIVE alert already exists for this zone
        IF EXISTS (
            SELECT 1 FROM alerts
            WHERE zone_id = NEW.zone_id
            AND status = 'ACTIVE'
        ) THEN
            RETURN NEW; -- alert already open, nothing to do
        END IF;
        -- Retrieve the most recent water level sensor for this zone
        -- to ensure alert traceability
        SELECT s.sensor_id INTO v_sensor_id
        FROM sensors s
        JOIN water_level_measurements w ON w.sensor_id = s.sensor_id
        WHERE s.zone_id = NEW.zone_id
        AND s.status = 'active' -- only active sensors
        ORDER BY w.timestamp DESC
        LIMIT 1;
        -- Insert the alert with full zone + index + sensor reference
        INSERT INTO alerts (zone_id, index_id, sensor_id, alert_date, alert_type, message, status)
        VALUES (NEW.zone_id, NEW.index_id, v_sensor_id, NOW(), 'FLOOD',
            FORMAT(
                'Critical flood risk detected. Zone: %s | Index: %s | Sensor: %s',
                NEW.zone_id,
                NEW.index_value,
                v_sensor_id
            ),
            'ACTIVE'
        );
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

(b) Function

Ce trigger est attaché à la table `risk_indices`. Il se déclenche après l’insertion (AFTER INSERT) d’un nouvel indice de risque.

Lorsqu’un indice est calculé, le trigger appelle la fonction `generate_critical_alert()` qui vérifie le niveau de risque.

Si le niveau est critique, une nouvelle alerte est automatiquement créée dans la table `alertes`.

Sinon aucune action n’est effectuée.

Ce mécanisme permet de générer automatiquement des alertes lorsque le système détecte un risque de crue.

– **Prompt 3 :**

Créer un Trigger qui ferme automatique l’alerte si le niveau de risque baisse.

– **Résultats :**

FIGURE 19 – Fermeture automatique d’alerte si niveau baisse

```
CREATE TRIGGER trg_close_alert
AFTER INSERT ON water_level_measurements
FOR EACH ROW
EXECUTE FUNCTION close_alert_if_low();
```

(a) Trigger

```
CREATE OR REPLACE FUNCTION close_alert_if_low()
RETURNS TRIGGER AS $$
DECLARE
    v_critical_level NUMERIC;
    v_zone_id INT;
BEGIN
    -- Retrieve the zone associated with the sensor send
    SELECT zone_id INTO v_zone_id
    FROM sensors
    WHERE sensor_id = NEW.sensor_id;

    -- Retrieve the critical threshold for this zone
    SELECT critical_level INTO v_critical_level
    FROM zones
    WHERE zone_id = v_zone_id;

    -- If the water level is below 50% of the threshold :
    -- Automatically resolve all active alerts for this :
    IF NEW.water_level_m < (v_critical_level * 0.5) THEN
        UPDATE alerts
        SET status = 'RESOLVED'
        WHERE zone_id = v_zone_id
        AND status = 'ACTIVE';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

(b) Function

On crée un trigger attaché à la table `water_level_measurements`. Il se déclenche automatiquement après l’insertion (AFTER INSERT) d’une nouvelle mesure de niveau d’eau. Son rôle est de vérifier si le niveau d’eau a suffisamment baissé pour clôturer une alerte active. Lorsqu’une nouvelle mesure est enregistrée, le trigger appelle la fonction `close_alert_if_low()`, dans ce cas, la fonction met à jour la table `alertes`.

– **Trigger recommandé :**

FIGURE 20 – Fermeture automatique d’alerte si niveau baisse

```
CREATE TRIGGER trg_alert_level_exceeded
AFTER INSERT ON water_level_measurements
FOR EACH ROW
EXECUTE FUNCTION fn_alert_level_exceeded();
```

(a) Trigger

```
CREATE OR REPLACE FUNCTION fn_alert_level_exceeded()
RETURNS TRIGGER AS $$
DECLARE
    v_zone_id INT;
    v_critical_level NUMERIC;
BEGIN
    SELECT s.zone_id, z.critical_level
    INTO v_zone_id, v_critical_level
    FROM sensors s
    JOIN zones z ON z.zone_id = s.zone_id
    WHERE s.sensor_id = NEW.sensor_id;

    IF NEW.water_level_m >= v_critical_level THEN
        IF NOT EXISTS (
            SELECT 1 FROM alerts
            WHERE zone_id = v_zone_id
            AND status = 'ACTIVE'
            AND alert_type = 'LEVEL_EXCEEDED'
        ) THEN
            INSERT INTO alerts (zone_id, sensor_id, alert_date, alert_type, message, status)
            VALUES (v_zone_id, NEW.sensor_id, NOW(), 'LEVEL_EXCEEDED',
                FORMAT(
                    'Critical level exceeded. Zone: %s | Measured level: %s m | Level: %s m | Sensor: %s',
                    v_zone_id,
                    NEW.water_level_m,
                    v_critical_level,
                    NEW.sensor_id
                ),
                'ACTIVE'
            );
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

(b) Function

Ce trigger permet d’automatiser la génération d’alertes dans le système dès qu’une nouvelle

mesure de niveau d'eau est insérée dans la table `water_level_measurements`. À chaque insertion, la fonction associée récupère la zone du capteur ainsi que son seuil critique, puis compare la valeur mesurée avec ce seuil. Si le niveau d'eau dépasse ou atteint le seuil critique, et qu'aucune alerte active similaire n'existe déjà, une nouvelle alerte est automatiquement créée dans la table `alerts`. L'intérêt principal de ce mécanisme est d'assurer une détection en temps réel des situations à risque directement au niveau de la base de données, sans dépendre de l'application, garantissant ainsi fiabilité, réactivité et cohérence des données. Ce trigger agit directement sur les mesures de niveau d'eau en comparant chaque valeur au seuil critique de la zone, permettant une alerte immédiate et locale. Le premier trigger, quant à lui, se base sur les indices de risque déjà calculés et déclenche une alerte uniquement lorsque le niveau de risque global est critique, ce qui permet une vision plus agrégée et décisionnelle du danger. Ainsi, les deux triggers sont complémentaires : l'un est réactif et basé sur les données brutes, l'autre est analytique et basé sur des résultats synthétiques.

– **Prompt 4 :**

Créer une fonction qui analyse la tendance du risque pour une zone sur une période donnée.

– **Résultats :**

FIGURE 21 – Analyse la tendance

```
CREATE OR REPLACE FUNCTION get_risk_trend(p_zone_id INT, p_start_date TIMESTAMP, p_end_date TIMESTAMP)
RETURNS TABLE (zone_id INT, start_index NUMERIC, end_index NUMERIC, trend TEXT)
LANGUAGE plpgsql AS $$
DECLARE
    v_start_index NUMERIC;
    v_end_index NUMERIC;
BEGIN
    -- First risk index value in the period (ascending order)
    SELECT index_value INTO v_start_index
    FROM risk_indices
    WHERE zone_id = p_zone_id
    AND calculation_date >= p_start_date
    AND calculation_date <= p_end_date
    ORDER BY calculation_date ASC
    LIMIT 1;

    -- Last risk index value in the period (descending order)
    SELECT index_value INTO v_end_index
    FROM risk_indices
    WHERE zone_id = p_zone_id
    AND calculation_date >= p_start_date
    AND calculation_date <= p_end_date
    ORDER BY calculation_date DESC
    LIMIT 1;

    -- Compare the two values to determine the trend
    RETURN QUERY
    SELECT
        p_zone_id,
        v_start_index,
        v_end_index,
        CASE
            WHEN v_end_index > v_start_index THEN 'increasing' -- risk is growing
            WHEN v_end_index < v_start_index THEN 'decreasing' -- risk is lowering
            ELSE 'stable' -- risk is constant
        END;
END;
$$;
```

L'idée est de comparer le premier indice de risque de la période avec le dernier pour déterminer si le risque est en augmentation, stable ou en diminution. La fonction reçoit la zone analysée, début de la période et fin de la période. Elle récupère le premier indice de risque de la période et le dernier indice de risque. Puis elle compare ces deux valeurs pour déterminer la tendance du risque : augmentation si le risque augmente, diminution si le risque baisse ou stable si le risque reste constant. Cette fonction est utile pour analyser l'évolution du risque d'inondation dans

une zone sur une période donnée et peut être utilisée dans un tableau de bord de surveillance hydrologique.

3.2.2 Procédures

– Prompt 3 :

Créer une procédure stockée pour calculer l'indice de risque de crue en croisant le niveau actuel et l'historique des pluies de 7 jours.

– Résultats :

FIGURE 22 – Calculer l'indice de risque de crue

```
CREATE OR REPLACE PROCEDURE calculate_flood_risk(p_zone_id INT)
LANGUAGE plpgsql AS $$
DECLARE
    v_current_level    NUMERIC;
    v_avg_rain         NUMERIC;
    v_critical_level   NUMERIC;
    v_max_rain         NUMERIC := 150; -- historical max Souss-Massa in
    v_level_norm       NUMERIC; -- normalized level [0, 1]
    v_rain_norm        NUMERIC; -- normalized rain [0, 1]
    v_index            NUMERIC;
    v_level            VARCHAR(20);
BEGIN
    -- Step 1: Retrieve the latest water level measurement for the zone
    SELECT m.water_level_m INTO v_current_level
    FROM water_level_measurements m
    JOIN sensors c ON m.sensor_id = c.sensor_id
    WHERE c.zone_id = p_zone_id
    AND c.status = 'active' -- ignore sensors out of service
    ORDER BY m.timestamp DESC
    LIMIT 1;

    -- Step 2: Retrieve the critical threshold for the zone
    SELECT critical_level INTO v_critical_level
    FROM zones
    WHERE zone_id = p_zone_id;
```

(a)

```
-- Step 3: Calculate the average rainfall over the last 7 days
-- c.status = 'active' filter added for consistency with step 1:
-- measurements from sensors out of service or in maintenance are excluded
SELECT COALESCE(AVG(mp.rain_mm), 0) INTO v_avg_rain
FROM rain_measurements mp
JOIN sensors c ON mp.sensor_id = c.sensor_id
WHERE c.zone_id = p_zone_id
AND c.status = 'active' -- ignore sensors out of service / in maintenance
AND mp.timestamp >= NOW() - INTERVAL '7 days';

-- Step 4: Normalization of variables between 0 and 1
-- Avoids the issue of different units (meters vs millimeters)
v_level_norm := LEAST(COALESCE(v_current_level, 0) / NULLIF(v_critical_level, 0), 1.0);
v_rain_norm := LEAST(COALESCE(v_avg_rain, 0) / v_max_rain, 1.0);

-- Step 5: Calculation of the weighted index
-- 60% water level: direct and immediate indicator of danger
-- 40% rain : predictive indicator (saturated soils amplify risk)
v_index := (v_level_norm * 0.6) + (v_rain_norm * 0.4);

-- Step 6: Classification of the risk level
IF v_index >= 0.9 THEN v_level := 'CRITICAL'; -- immediate intervention
ELSEIF v_index >= 0.7 THEN v_level := 'HIGH'; -- reinforced surveillance
ELSEIF v_index >= 0.4 THEN v_level := 'MEDIUM'; -- vigilance
ELSE v_level := 'LOW'; -- normal situation
END IF;

-- Step 7: Save the result in the history
INSERT INTO risk_indices(zone_id, calculation_date, index_value, risk_level)
VALUES (p_zone_id, NOW(), v_index, v_level);

END;
$;$
```

(b)

La procédure stockée calcule un indice de risque de crue en combinant le niveau actuel de l'eau mesuré par les capteurs et l'historique récent des précipitations. Le niveau actuel correspond à la dernière mesure disponible, tandis que l'historique des pluies est calculé sur les sept derniers jours afin de prendre en compte l'effet cumulatif des précipitations sur le risque de crue.

– Prompt 3 :

Créer une procédure qui permet de déplacer les anciennes mesures vers les tables d'archive.

– Résultats :

FIGURE 23 – Archivé anciennes mesures

```
CREATE OR REPLACE PROCEDURE archive_old_measurements(p_cutoff_date TIMESTAMP)
LANGUAGE plpgsql AS $$
BEGIN
    -- Archiving water level measurements prior to the cutoff date
    INSERT INTO water_level_measurements_archive
    SELECT * FROM water_level_measurements
    WHERE timestamp < p_cutoff_date;

    -- Deleting archived data from the active table
    DELETE FROM water_level_measurements
    WHERE timestamp < p_cutoff_date;

    -- Archiving rain measurements prior to the cutoff date
    INSERT INTO rain_measurements_archive
    SELECT * FROM rain_measurements
    WHERE timestamp < p_cutoff_date;

    -- Deleting archived data from the active table
    DELETE FROM rain_measurements
    WHERE timestamp < p_cutoff_date;

    RAISE NOTICE 'Archiving complete: measurements prior to % moved.', p_cutoff_date;
END;
$$;
```

Cette procédure permet d’optimiser les performances de la base, gérer un grand volume de données de capteurs et conserver l’historique des mesures pour simuler un vrai système de gestion de données environnementales.

- **Hallucination** : La procédure ne se déclenche pas automatiquement.
- **Solution** : Utiliser un scheduler

```
CREATE EXTENSION IF NOT EXISTS pg_cron;

-- Schedule archive job (every day at midnight)
SELECT cron.schedule(
    'archive_job',
    '0 0 * * *',
    $$CALL archive_old_measurements(NOW() - INTERVAL '6 months');$$
);
```

C’est un planificateur automatique permet d’exécute une requête automatiquement à minuit. Permet archive toutes les données plus anciennes que 6 mois.

3.3 Présentation du projet

Cette section présente les différentes interfaces de l'application **Oued-Souss Alert**. Elle permet de visualiser concrètement le fonctionnement du système à travers les écrans principaux du tableau de bord et des modules de gestion des données hydrologiques.

Chaque interface a été conçue pour répondre à un besoin spécifique : supervision des zones, affichage des alertes, consultation des mesures et suivi des risques en temps réel.

La figure suivante représente la page d'accueil de l'application. Elle permet à l'utilisateur d'accéder aux principales fonctionnalités du système et constitue le point d'entrée du tableau de bord.

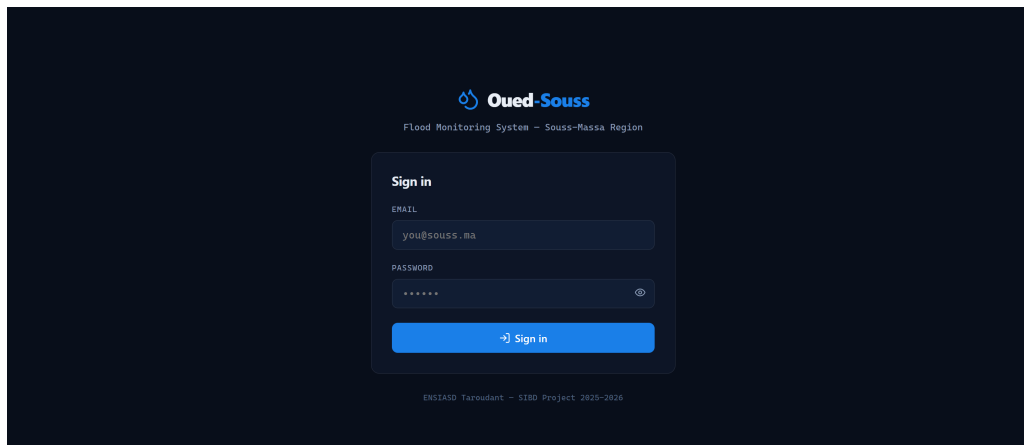


FIGURE 24 – Page d'accueil

La figure suivante présente le tableau de bord principal qui affiche une vue globale des zones surveillées ainsi que leur état en temps réel.

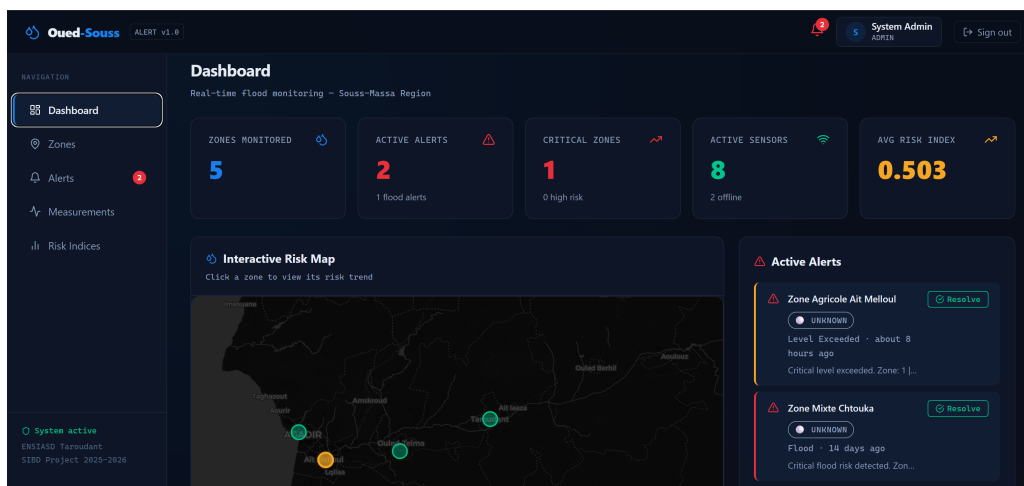


FIGURE 25 – Tableau de bord - Vue générale 1

Cette interface du tableau de bord permet de visualiser des indicateurs supplémentaires liés à l'état des zones et aux risques détectés.

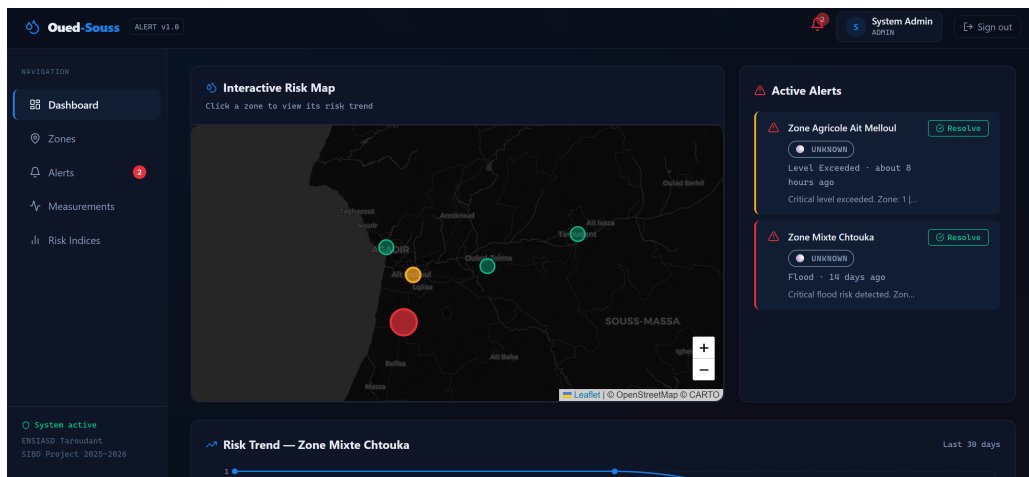


FIGURE 26 – Tableau de bord - Vue détaillée 2

La figure suivante illustre une autre vue du tableau de bord permettant de suivre l'évolution des alertes et des données en temps réel.



FIGURE 27 – Tableau de bord - Suivi des alertes

Cette interface est dédiée à la gestion des zones géographiques surveillées par le système, avec leurs caractéristiques et leurs niveaux de risque.

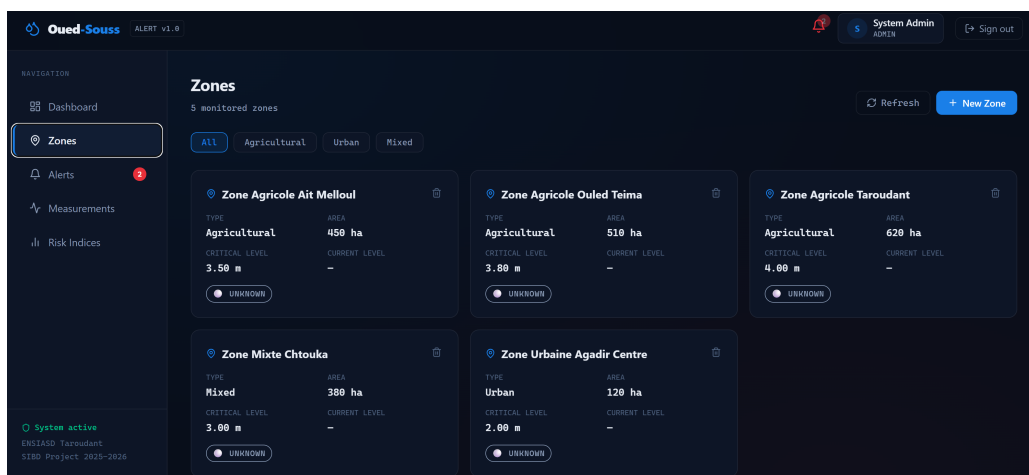


FIGURE 28 – Gestion des zones

La figure suivante présente la liste des alertes générées automatiquement par le système en fonction des niveaux de risque détectés.

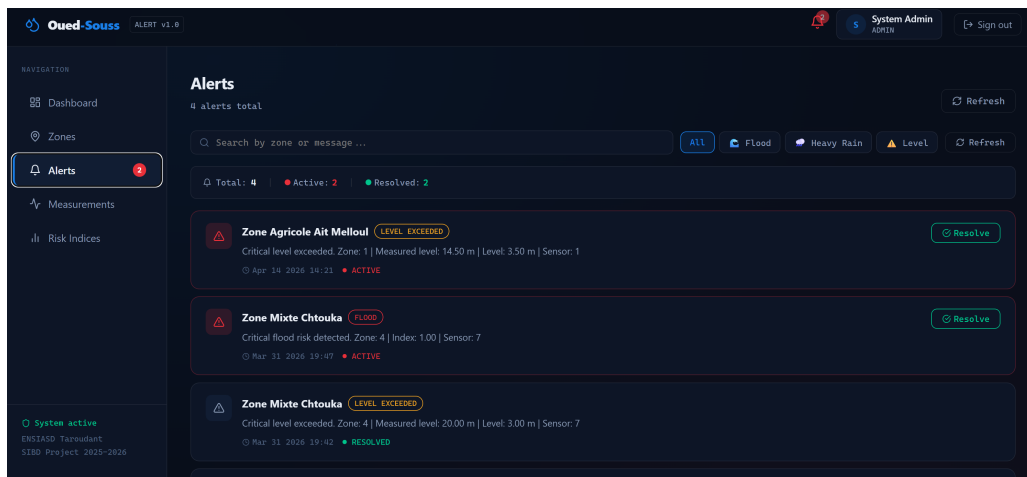


FIGURE 29 – Gestion des alertes

Cette interface permet de visualiser les mesures du niveau d'eau enregistrées par les capteurs installés sur le terrain.

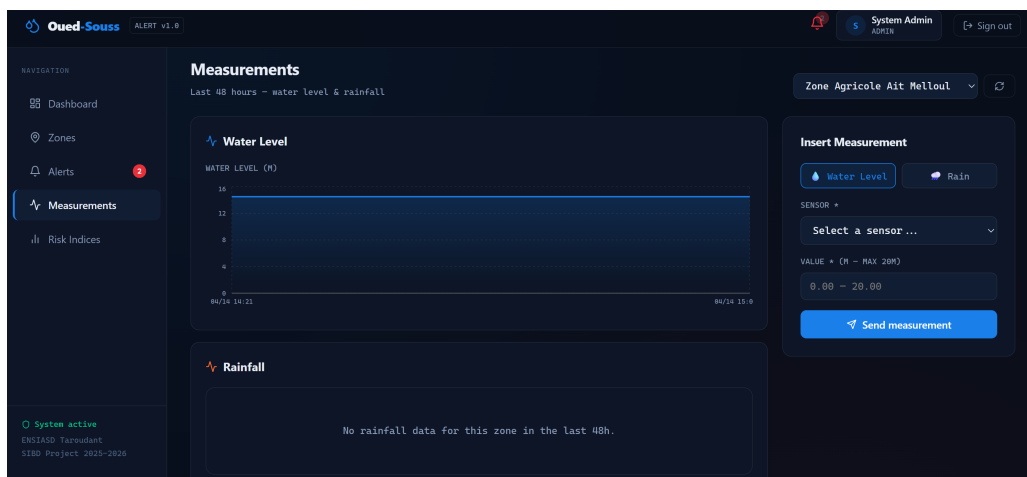


FIGURE 30 – Mesures du niveau d'eau

La figure suivante présente les mesures des précipitations collectées par les capteurs météorologiques.

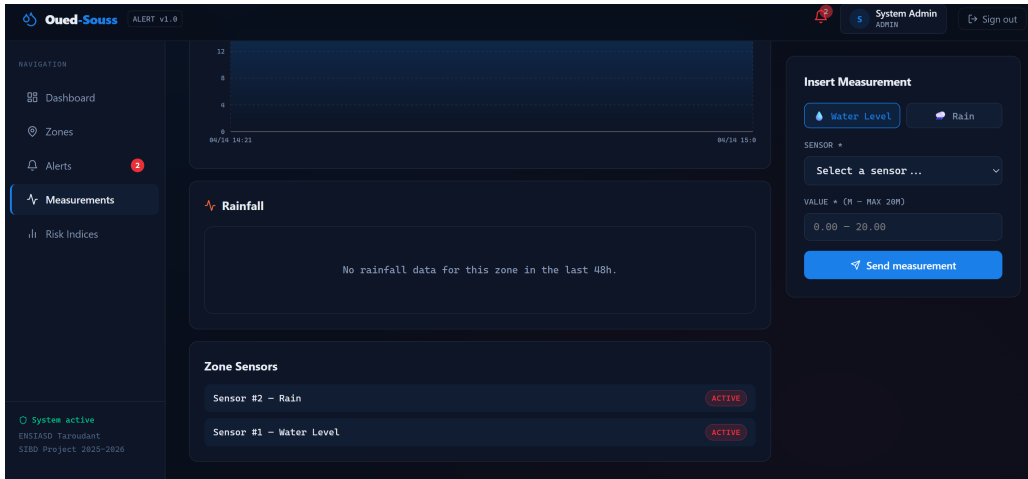


FIGURE 31 – Mesures de pluie

Enfin, cette interface affiche les indices de risque calculés automatiquement pour chaque zone afin d'évaluer le niveau de danger.



FIGURE 32 – Indice de risque

4 Conclusion

Ce projet a permis de concevoir et de développer un système intelligent de surveillance et d'alerte des crues agricoles basé sur une architecture moderne et performante.

L'intégration des mécanismes d'automatisation au niveau de la base de données, notamment les triggers et les procédures stockées, a permis d'assurer une détection rapide et fiable des situations à risque.

Le système développé offre une solution efficace pour anticiper les inondations et améliorer la prise de décision grâce à une visualisation claire des données et des alertes.

Ce travail constitue une base solide pour le développement futur de systèmes de surveillance environnementale plus avancés intégrant des données réelles et des modèles prédictifs plus complexes.